

**SIMATS ENGINEERING
ASSESSMENT TOOL 3**

**COURSE NAME: CRYPTOGRAPHY AND NETWORK
SECURITY COURSE CODE: CSA51**

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS:

Total Marks: 50

Annexure A

Code of Conduct:

*I **T.GOKULNATH(192521139)**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: T.Gokulnath

Annexure A

Debugging & Optimisation Task

1. Debugging Hash Function Implementation (SHA Family)

A Python program implementing a SHA-based hashing algorithm produces inconsistent hash outputs for the same input.

Identify errors in message padding and block processing steps.

Fix issues related to bitwise operations and endianness handling.

Optimize the implementation to improve processing speed for large files (≥ 50 MB).

Evaluate how secure hash functions improve integrity compared to basic checksum methods.

Problem Identification

- The file is opened in text mode ('r'), so Python performs newline translation and applies platform/locale-dependent text decoding before the bytes are hashed — the same file can therefore hash differently on Windows vs. Linux, or after any trailing-newline edit.
- The SHA-256 hasher object is re-created inside the read loop, so every previous update() call is discarded — only the final byte read ever contributes to the digest, which explains the inconsistent output.
- Reading the file one byte at a time makes the implementation impractically slow for files ≥ 50 MB.

Debugging — Faulty Code

```
import hashlib

def hash_file(filepath):
    hasher = hashlib.sha256()
    with open(filepath, 'r') as f:      # BUG: text mode -> newline
        data = f.read(1)               # translation + platform-dependent
        while data:                    # encoding changes the byte stream
            hasher = hashlib.sha256()   # BUG: re-created every loop, so
            hasher.update(data.encode()) # only the LAST byte is ever hashed
            data = f.read(1)            # BUG: 1-byte reads -> extremely slow
    return hasher.hexdigest()
```

Debugging & Optimisation — Corrected Code

```
import hashlib

def hash_file(filepath, chunk_size=65536):
    hasher = hashlib.sha256()          # initialised ONCE, outside the loop
    with open(filepath, 'rb') as f:    # binary mode -> byte-exact, portable
        while True:
            chunk = f.read(chunk_size) # 64 KB buffered reads -> fast for
            if not chunk:               # files  $\geq 50$  MB, low memory footprint
                break
            hasher.update(chunk)
    return hasher.hexdigest()
```

Optimization

Switching to binary mode removes the platform-dependent text decoding, guaranteeing the same byte stream (and therefore the same hash) on every run. Reading in 64 KB buffered chunks instead of 1 byte at a time reduces the number of Python-level `read()` calls by roughly 65,000×, which is the dominant cost for large files — this brings a 50 MB file's hashing time down from several minutes to well under a second, since the underlying C implementation of SHA-256 in `hashlib` does the heavy lifting once enough data is handed to it per call.

Analysis — Hash Functions vs. Basic Checksums

A checksum such as CRC32 is designed only to catch accidental transmission or storage errors; it is fast but linear and trivially invertible/forgable — an attacker can deliberately craft a modified file with the same CRC32 value in a fraction of a second. A cryptographic hash function like SHA-256 is designed to resist this: it is collision-resistant (computationally infeasible to find two inputs with the same digest), pre-image resistant, and exhibits the avalanche effect, where changing a single bit of input changes roughly half the output bits unpredictably. This makes SHA-256 suitable for security-critical integrity verification (e.g., detecting deliberate tampering), whereas CRC32 is only appropriate for detecting unintentional corruption such as network noise.

2. Optimization of Secure Socket Layer (SSL/TLS) Simulation

A Python-based simulation of SSL/TLS handshake shows delays and occasional handshake failures.

Debug issues in certificate validation and key exchange steps.

Optimize the handshake process to reduce latency.

Refactor the code to reuse session keys efficiently (session resumption).

Analyze the performance vs security trade-offs in TLS optimization.

Problem Identification

- `validate_certificate()` unconditionally returns `True`, so expired or forged certificates are always accepted — a textbook opening for man-in-the-middle attacks.
- The Diffie–Hellman key exchange uses a toy 5-bit prime ($p = 23$); besides being trivially brute-forceable, such a small modulus also causes handshake failures when exponent values exceed the group's usable range.
- A brand-new handshake (fresh random keys) is performed on every single call, even for a client that connected moments earlier — this is the direct cause of the observed latency.

Debugging — Faulty Code

```
import random

class TLShandshake:
    def __init__(self):
        self.sessions = {}

    def validate_certificate(self, cert):
        return True          # BUG: no expiry/sig check

    def key_exchange(self, client_pub, server_priv):
        p = 23                # BUG: toy DH prime, brute-forceable
        shared = pow(client_pub, server_priv, p)
        return shared
```

```

def handshake(self, client_id, cert):
    if not self.validate_certificate(cert):
        raise Exception("Invalid cert")
    server_priv = random.randint(1, 100) # BUG: redone every call,
    shared_key = self.key_exchange(      # even for a repeat client
        random.randint(1, 100), server_priv)
    return shared_key

```

Debugging & Optimisation — Corrected Code

```

import os, hmac, hashlib, time

class TLSHandshake:
    DH_PRIME = int("FFFFFFFFFFFFFFFFC90FDAA22168C234C4C6628B80DC1CD"
        "129024E088A67CC74020BBEA63B139B22514A08798E3404"
        "DDEF9519B3CD3A431B302B0A6DF25F14374FE1356D6D51C", 16)
    DH_GEN = 2

    def __init__(self, session_ttl=3600):
        self.sessions = {}
        self.session_ttl = session_ttl # max reuse window for a session

    def validate_certificate(self, cert):
        if cert["expiry"] < time.time():
            return False
        expected_sig = hmac.new(cert["ca_key"], cert["subject"].encode(),
            hashlib.sha256).hexdigest()
        return hmac.compare_digest(expected_sig, cert["signature"])

    def key_exchange(self, client_priv, server_priv):
        server_pub = pow(self.DH_GEN, server_priv, self.DH_PRIME)
        client_pub = pow(self.DH_GEN, client_priv, self.DH_PRIME)
        return pow(client_pub, server_priv, self.DH_PRIME)

    def handshake(self, client_id, cert):
        cached = self.sessions.get(client_id)
        if cached and time.time() - cached["ts"] < self.session_ttl:
            return cached["key"] # session resumption -> skips full handshake

        if not self.validate_certificate(cert):
            raise ValueError("Certificate validation failed")

        client_priv = int.from_bytes(os.urandom(32), "big")
        server_priv = int.from_bytes(os.urandom(32), "big")
        shared_key = self.key_exchange(client_priv, server_priv)
        self.sessions[client_id] = {"key": shared_key, "ts": time.time()}
        return shared_key

```

Optimization

Certificate validation now checks both expiry and an HMAC-based signature using constant-time comparison (`hmac.compare_digest`), closing the MITM gap. Replacing the 5-bit prime with a standard 2048-bit MODP Diffie–Hellman group (RFC 3526) removes both the brute-force weakness and the exponent-range failures. Most importantly, session resumption caches the

derived shared key per client_id for a bounded TTL, so a returning client within that window skips the expensive asymmetric key exchange entirely and reuses the cached key — this is the main latency win, cutting a repeat handshake from a full DH exchange down to a single dictionary lookup.

Analysis — Performance vs. Security Trade-off

Session resumption is the classic TLS optimisation trade-off: it removes the cost of a fresh Diffie–Hellman exchange and certificate check on every reconnect, which is the main source of handshake latency, but it also means the same shared key is reused for the lifetime of the session cache. If the TTL is set too long, a compromise of that cached key exposes a longer window of traffic and weakens perfect forward secrecy compared to a fresh key per session. The corrected implementation balances this by bounding session_ttl (here 3600 s) so that performance gains from resumption are capped by a periodic forced re-handshake, which is the same principle real TLS 1.3 session tickets and 0-RTT resumption rely on.

3. Debugging and Optimizing Digital Signature Algorithm (DSA)

A Python implementation of DSA generates invalid signatures during verification.

Identify issues in parameter selection and random number generation.

Fix errors in signature generation and verification steps.

Optimize modular arithmetic operations for faster execution.

Evaluate the impact of randomness quality on signature security.

Problem Identification

- The per-signature random nonce k is hard-coded to 5 instead of being freshly and randomly generated — reusing k across signatures is a catastrophic DSA flaw.
- `random.randint` would also be inappropriate for k even if not hard-coded, because Python's random module is a Mersenne Twister PRNG, not cryptographically secure and therefore predictable.
- `dsa_verify()` never checks that $0 < r < q$ and $0 < s < q$ before using them, so malformed or maliciously crafted (r, s) pairs can cause incorrect verification results.

Debugging — Faulty Code

```
def dsa_sign(msg_hash, x, p, q, g):
    k = 5 # BUG: fixed/reused nonce -> leaks x
    r = pow(g, k, p) % q
    k_inv = pow(k, q - 2, q)
    s = (k_inv * (msg_hash + x * r)) % q
    return (r, s)

def dsa_verify(msg_hash, r, s, y, p, q, g):
    w = pow(s, q - 2, q) # BUG: no range check on r, s
    u1 = (msg_hash * w) % q
    u2 = (r * w) % q
    v = ((pow(g, u1, p) * pow(y, u2, p)) % p) % q
    return v == r
```

Debugging & Optimisation — Corrected Code

```

import secrets

def dsa_sign(msg_hash, x, p, q, g):
    while True:
        k = secrets.randbelow(q - 1) + 1 # fresh CSPRNG nonce
        r = pow(g, k, p) % q             # fast built-in modexp
        if r == 0:
            continue
        k_inv = pow(k, q - 2, q)
        s = (k_inv * (msg_hash + x * r)) % q
        if s != 0:
            return (r, s)

def dsa_verify(msg_hash, r, s, y, p, q, g):
    if not (0 < r < q) or not (0 < s < q): # domain check
        return False
    w = pow(s, q - 2, q)
    u1 = (msg_hash * w) % q
    u2 = (r * w) % q
    v = ((pow(g, u1, p) * pow(y, u2, p)) % p) % q
    return v == r

```

Optimization

Modular exponentiation (`pow(g, k, p)`, `pow(s, q-2, q)`, etc.) already uses Python's built-in three-argument `pow()`, which invokes an optimised C-level square-and-multiply implementation rather than a naïve Python loop — this is significantly faster than a hand-written modular exponentiation loop and is retained/relied on in both the signing and verification paths, so no further change was needed there beyond ensuring it is used consistently for every modular power computed.

Analysis — Impact of Randomness Quality on Security

DSA security depends entirely on k being unique, uniformly random, and secret for every signature. If the same k is reused across two signatures (r_1, s_1) and (r_2, s_2) on different messages, an attacker can algebraically solve two linear equations in the two unknowns k and the private key x , recovering the private key completely — this is precisely the bug in the faulty implementation, which hard-codes $k = 5$ for every call. Even without exact reuse, a predictable PRNG (such as Python's `random` module, which is not cryptographically secure) can let an attacker narrow down or reconstruct k from observed signatures. The fix uses `secrets.randbelow()`, a CSPRNG, to guarantee a fresh, unpredictable k for every signature, which is the property the entire security of DSA rests on.

4. Debugging Key Management System

A Python-based key management system fails to securely store and retrieve cryptographic keys.

Identify flaws in key storage (plaintext storage, improper encryption).

Fix issues related to key lifecycle management (generation, distribution, rotation).

Optimize secure storage using encryption and access control mechanisms.

Analyze how poor key management can compromise an entire cryptographic system.

Problem Identification

- Generated keys are written straight to a JSON file with no encryption — anyone with file-system read access (or a backup, or a leaked repository) obtains every key in the store.
- There is no key lifecycle management: no creation timestamp, no expiry, and no rotation mechanism — a key generated once can be used forever, unbounded.
- There is no access control or master-secret separation — the store has no concept of a password or derived key protecting the file's contents.

Debugging — Faulty Code

```
import json, os

class KeyStore:
    def __init__(self, path="keys.json"):
        self.path = path
        self.keys = {}

    def generate_key(self, key_id):
        key = os.urandom(32)
        self.keys[key_id] = key.hex() # BUG: stored as plain hex
        self._save()

    def _save(self):
        with open(self.path, "w") as f:
            json.dump(self.keys, f) # BUG: whole store in PLAINTEXT

    def get_key(self, key_id):
        return self.keys.get(key_id) # BUG: no expiry / rotation check
```

Debugging & Optimisation — Corrected Code

```
import json, os, time, base64
from cryptography.fernet import Fernet
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC
from cryptography.hazmat.primitives import hashes

class KeyStore:
    def __init__(self, path, master_password, salt, rotation_days=90):
        self.path, self.rotation_days = path, rotation_days
        kdf = PBKDF2HMAC(algorithm=hashes.SHA256(), length=32,
                          salt=salt, iterations=200_000)
        master_key = base64.urlsafe_b64encode(kdf.derive(master_password.encode()))
        self.fernet = Fernet(master_key) # keys encrypted at rest; master key never stored
        self.keys = self._load()

    def generate_key(self, key_id):
        key = os.urandom(32)
        self.keys[key_id] = {"key": key.hex(), "created": time.time()}
        self._save()
        return key

    def rotate_key(self, key_id):
        return self.generate_key(key_id) # explicit lifecycle: generate -> use -> rotate
```

```

def get_key(self, key_id):
    entry = self.keys.get(key_id)
    if not entry:
        return None
    if (time.time() - entry["created"]) / 86400 > self.rotation_days:
        raise PermissionError("Key expired; call rotate_key() first")
    return bytes.fromhex(entry["key"])

def _save(self):
    blob = self.fernet.encrypt(json.dumps(self.keys).encode())
    with open(self.path, "wb") as f:
        f.write(blob)

def _load(self):
    if not os.path.exists(self.path):
        return {}
    with open(self.path, "rb") as f:
        return json.loads(self.fernet.decrypt(f.read()).decode())

```

Optimization

The key store file is now encrypted at rest using Fernet (AES-128 in CBC mode with an HMAC), where the encryption key is derived from a master password via PBKDF2-HMAC-SHA256 with 200,000 iterations rather than being stored anywhere on disk. Each key entry also records a creation timestamp, and `get_key()` enforces a `rotation_days` limit, forcing `rotate_key()` before an aged key can be reused — turning an unbounded, static key store into one with an explicit, enforced lifecycle.

Analysis — Impact of Poor Key Management

Cryptographic algorithms such as AES or RSA can be mathematically unbreakable in practice, yet the entire system's security still collapses if the keys protecting them are mismanaged. Plaintext key storage turns a single file-system compromise (a misconfigured backup, a leaked repository, an insider) into a total breach of every piece of data those keys protect, regardless of how strong the encryption algorithm itself is. Similarly, keys with no expiry or rotation stay valid indefinitely, so a key exposed once (even briefly) remains a permanent liability rather than a bounded one. This illustrates a core principle of applied cryptography: the strength of a cryptosystem is only as strong as its weakest link, which in practice is almost always key management rather than the cipher itself.

5. Optimization of Secure File Transfer Protocol (SFTP-like System)

A Python script simulating secure file transfer experiences slow transfer speeds and occasional data corruption.

Debug issues in encryption/decryption synchronization.

Fix errors in packet handling and integrity verification.

Optimize throughput for large file transfers (≥100 MB).

Compare the optimized system with insecure file transfer methods and justify the improvements.

Problem Identification

- AES is used in ECB mode, which encrypts identical plaintext blocks to identical ciphertext blocks — this leaks structural patterns in the file and is not semantically secure.
- The entire file is read into memory and encrypted/decrypted in a single call, which is slow and memory-heavy for files ≥ 100 MB and offers no way to process data as a stream.
- There is no integrity or authentication check anywhere in the pipeline, so any bit-flip, truncation, or deliberate tampering in transit silently produces corrupted plaintext with no error raised — matching the reported 'occasional data corruption' symptom.

Debugging — Faulty Code

```
from Crypto.Cipher import AES

def encrypt_file(path, key):
    cipher = AES.new(key, AES.MODE_ECB) # BUG: ECB leaks patterns
    with open(path, "rb") as f:        # BUG: whole file in memory
        data = f.read()                # slow/high RAM for >=100MB
    pad_len = 16 - (len(data) % 16)
    data += bytes([pad_len]) * pad_len
    return cipher.encrypt(data)

def decrypt_file(ciphertext, key):
    cipher = AES.new(key, AES.MODE_ECB)
    data = cipher.decrypt(ciphertext) # BUG: no integrity check ->
    pad_len = data[-1]                # corruption goes undetected
    return data[:-pad_len]
```

Debugging & Optimisation — Corrected Code

```
from Crypto.Cipher import AES
from Crypto.Util import Counter
import hmac, hashlib, os

CHUNK = 1024 * 1024 # 1 MB streamed chunks -> high throughput, constant memory

def encrypt_file(path, key, hmac_key, out_path):
    nonce = os.urandom(8)
    cipher = AES.new(key, AES.MODE_CTR, counter=Counter.new(64, prefix=nonce))
    mac = hmac.new(hmac_key, digestmod=hashlib.sha256)

    with open(path, "rb") as fin, open(out_path, "wb") as fout:
        fout.write(nonce)
        while True:
            chunk = fin.read(CHUNK)
            if not chunk:
                break
            enc_chunk = cipher.encrypt(chunk)
            mac.update(enc_chunk)
            fout.write(enc_chunk)
        fout.write(mac.digest()) # HMAC tag appended -> integrity verification

def decrypt_file(in_path, key, hmac_key, out_path):
```

```
with open(in_path, "rb") as fin:
    nonce, body = fin.read(8), fin.read()
    ciphertext, tag = body[:-32], body[-32:]
    if not hmac.compare_digest(
        hmac.new(hmac_key, ciphertext, hashlib.sha256).digest(), tag):
        raise ValueError("Integrity check failed - file corrupted or tampered")

    cipher = AES.new(key, AES.MODE_CTR, counter=Counter.new(64, prefix=nonce))
    with open(out_path, "wb") as fout:
        for i in range(0, len(ciphertext), CHUNK):
            fout.write(cipher.decrypt(ciphertext[i:i + CHUNK]))
```

Optimization

Switching from ECB to CTR mode with a fresh random nonce per file removes the block-pattern leakage and, being a stream cipher mode, naturally supports chunked processing without padding. The file is now streamed through in fixed 1 MB chunks rather than loaded wholesale into memory, which keeps memory usage constant regardless of file size and lets the OS overlap disk I/O with encryption, substantially improving throughput for files ≥ 100 MB. An HMAC-SHA256 tag is computed incrementally over the ciphertext as it is produced and verified with a constant-time comparison before decryption proceeds, catching any corruption or tampering immediately instead of silently returning garbage plaintext.

Analysis — Comparison with Insecure File Transfer

A plain, insecure transfer (e.g., classic FTP) sends file contents unencrypted and includes no integrity mechanism at all, so it is fully exposed to eavesdropping and undetectable in-transit tampering, and any corruption is discovered only if the receiving application happens to fail later. The optimized system improves on this on all three fronts simultaneously: confidentiality (AES-CTR keeps contents unreadable and free of ECB pattern leakage), integrity (the appended HMAC tag makes any tampering or corruption immediately detectable before the corrupted data is ever used), and performance (chunked streaming keeps memory bounded and throughput high even for very large files) — demonstrating that security and performance are not inherently in conflict when the underlying primitives are chosen and applied correctly.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation